

横向联邦学习实验报告

学号：3230104177
姓名：陈奕诺

1. 实验原理

1.1 联邦学习的核心思想

联邦学习的目标是在不集中收集客户端原始数据的前提下，训练一个共享模型。每个客户端保留自己的本地数据，只接收服务器下发的全局模型，在本地完成训练后上传模型参数更新或参数差值。服务器对多个客户端上传的更新进行聚合，得到新的全局模型，再进入下一轮训练。

联邦学习包含以下核心概念：

- 分布式训练：在各个客户端本地进行训练，服务器只负责分发模型、收集模型参数或参数更新。
- 数据隐私保护：由于训练数据不上传到服务器，可以减少原始数据直接传输带来的隐私泄露风险。

联邦学习的关键步骤是：

- 初始化全局模型，由服务器端创建初始模型参数
- 模型分发，服务器将当前全局模型发送给被选中的客户端
- 每个客户端使用自己的本地数据训练，得到本地模型参数
- 客户端上传训练后的模型参数或参数更新
- 服务器收集多个客户端的本地模型参数，并使用 FedAvg 等方法聚合
- 服务器得到新的全局模型，用于下一轮通信或最终测试

与传统集中式学习相比，联邦学习不要求把所有数据上传到中心服务器，因此可以降低数据泄露风险，更适合存在隐私约束或数据分散存储的场景。集中式学习的优势是训练流程简单、数据分布统一、收敛稳定；联邦学习的优势是保护本地数据，但同时会带来通信开销、客户端数据非 IID、客户端算力不一致等问题。

1.2 FedAvg 算法

本实验使用 FedAvg 风格的聚合方法。每一轮中，服务器先从所有客户端中随机选择 k 个参与训练的客户端，然后将当前全局模型参数 w_t 下发给这些客户端。客户端使用本地数据训练若干轮，得到本地模型参数 w_t^i ，并上传参数差值：

$$\Delta w_i = w_t^i - w_t$$

服务器收集多个客户端的参数差值后进行聚合：

$$w_{t+1} = w_t + \lambda \sum_{i \in S_t} \Delta w_i$$

其中 S_t 是第 t 轮被选中的客户端集合， λ 是聚合系数。 λ 越大，客户端更新对全局模型影响越强； λ 过大可能导致训练不稳定，过小则可能收敛较慢。

1.3 参数含义

本实验重点比较每轮参与客户端数 k 的影响。 k 越大，每一轮聚合时使用的客户端数据越多，更新方向通常更稳定，但通信和本地训练开销也更大。其他主要参数包括：

参数	含义
no_models	总客户端数量
k	每轮参与聚合的客户端数量
global_epochs	全局训练轮数
local_epochs	每个客户端本地训练轮数
lambda	服务端聚合系数
lr	客户端本地训练学习率

2. 代码理解与实现

2.1 整体结构

实验代码主要由 `main.py`、`server.py`、`client.py`、`datasets.py` 和 `models.py` 组成。

`main.py` 负责读取配置文件、加载数据集、初始化服务器和客户端，并执行全局训练循环。每个全局轮次中，它随机选择 `k` 个客户端，初始化 `weight_accumulator`，依次调用客户端本地训练函数，将客户端上传的参数差值累加，最后调用服务器聚合并评估全局模型。

`server.py` 中的 `Server` 类维护全局模型和测试集加载器。`model_aggregate()` 遍历全局模型的每一层参数，将累积参数更新乘以 `lambda` 后加到全局模型上。`model_eval()` 在测试集上计算平均交叉熵损失和分类准确率。

`client.py` 中的 `Client` 类负责本地数据划分和本地训练。每个客户端根据自己的 `id` 从总训练集中取一段子集，使用 `SubsetRandomSampler` 构造本地 `DataLoader`。`local_train()` 先复制服务器全局模型参数，再用本地数据训练，最后返回本地模型与原全局模型之间的参数差值。

`datasets.py` 负责加载数据集。本实验使用 CIFAR-10，训练集使用随机裁剪、随机水平翻转、归一化等增强和预处理，测试集使用张量化与归一化。

`models.py` 负责根据配置创建模型。本实验使用 ResNet18，并将最后的全连接层输出修正为 10 类，以适配 CIFAR-10。

2.2 关键实现说明

全局训练的代码段如下：

```
for e in range(conf["global_epochs"]):
    candidates = random.sample(clients, conf["k"])

    weight_accumulator = {}

    for name, params in server.global_model.state_dict().items():
        weight_accumulator[name] = torch.zeros_like(params)

    for client in candidates:
        diff = client.local_train(server.global_model)

        for name, params in server.global_model.state_dict().items():
            weight_accumulator[name].add_(diff[name])

    server.model_aggregate(weight_accumulator)

    acc, loss = server.model_eval()
```

- `candidates`，每一轮从所有客户端随机选择 `k` 个参与训练。`weight_accumulator` 用于保存本轮所有客户端上传的参数插值综合。
- `for name, params in server.global_model.state_dict().items()` 代码为全局模型中的每一个参数层初始化一个同形状的零张量。例如卷积层权重、BatchNorm 参数、全连接层权重等都会在 `weight_accumulator` 中有对应位置。后续每个客户端上传的参数差值会累加到这些张量中。
- `for name, params in server.global_model.state_dict().items()` 将当前客户端上传的每一层参数更新累加到 `weight_accumulator` 中。

客户端本地训练的核心流程是：复制全局参数、本地 SGD 更新、计算参数差值。这样服务器不需要直接访问客户端数据，只需要接收参数更新。

```

def local_train(self, model):
    for name, param in model.state_dict().items():
        self.local_model.state_dict()[name].copy_(param.clone())

    optimizer = torch.optim.SGD(
        self.local_model.parameters(),
        lr=self.conf["lr"],
        momentum=self.conf["momentum"],
    )

    self.local_model.train()

    for e in range(self.conf["local_epochs"]):
        for batch_id, batch in enumerate(self.train_loader):
            data, target = batch

            if torch.cuda.is_available():
                data = data.cuda()
                target = target.cuda()

            optimizer.zero_grad()
            output = self.local_model(data)
            loss = torch.nn.functional.cross_entropy(output, target)
            loss.backward()
            optimizer.step()

        diff = {}

        for name, param in self.local_model.state_dict().items():
            diff[name] = param - model.state_dict()[name]

    return diff

```

- `optimizer` 定义了本地训练优化器。`lr` 是本地学习率，控制每一步参数更新幅度；`momentum` 是动量项，用于减少 SGD 更新过程中的震荡，使训练过程更稳定。
- 本地训练结束后，客户端计算参数差值 `diff`，代表客户端本地训练后的模型参数与服务器下发的全局模型参数之间的差。

服务端聚合的核心流程是：对每个参数层执行：

```

def model_aggregate(self, weight_accumulator):
    for name, data in self.global_model.state_dict().items():
        update_per_layer = weight_accumulator[name] * self.conf["lambda"]
        data.add_(update_per_layer)

```

- `model_aggregate` 函数定义了 FedAvg 聚合方式。`weight_accumulator[name]` 是本轮所有被选中客户端在该层参数上的更新总和。乘以 `lambda` 后得到服务器真正应用到全局模型上的更新量。

从实现逻辑上看，本实验代码符合 FedAvg 风格联邦学习流程。客户端首先复制服务器全局模型，在本地数据上独立训练，然后上传本地模型与全局模型之间的参数差值；服务器收集多个客户端的参数差值后，根据聚合系数 `lambda` 更新全局模型。整个过程中，服务器没有直接访问客户端原始训练数据，只使用模型参数更新完成全局训练。

该实现与 FedAvg 的核心思想一致：每轮训练中，全局模型通过多个客户端本地更新的平均或加权和进行更新。虽然本实验代码中使用的是 `lambda` 控制累积更新强度，而不是严格按照每个客户端样本数进行加权平均，但在客户端数据量大致相同的情况下，这种实现可以近似模拟 FedAvg 聚合过程，并用于观察客户端数量 `k`、全局轮数和本地训练轮数对训练效果的影响。

3. 实验过程

实验流程如下：

1. 加载 CIFAR-10 训练集和测试集。
2. 初始化一个服务器，服务器持有全局 ResNet18 模型。
3. 初始化 10 个客户端，每个客户端获得训练集的一个子集。
4. 每一轮全局训练随机选择 k 个客户端。
5. 被选中客户端复制当前全局模型，在本地训练 1 个 epoch。
6. 客户端上传本地模型与全局模型的参数差值。

- 7. 服务器累加客户端更新，并根据 $\lambda=0.1$ 更新全局模型。
- 8. 在测试集上评估全局模型准确率和损失。
- 9. 重复 10 轮，记录每轮结果。

本实验参数设置如下：

参数	实验取值	作用
global_epochs	10	全局通信轮数，即服务器聚合模型的轮数
local_epochs	1	每个客户端在本地数据上训练的轮数
k	3、5	每轮参与训练和上传参数的客户端数量
lambda	0.1	聚合系数，控制全局模型更新强度
lr	0.001	本地训练的初始学习率

- $\text{global_epochs} = 10$ ，可以在有限运行时间内观察模型准确率和损失值的变化趋势。训练轮数越高，预期准确率越高，损失值越低。
- $k = 3、5$ ，可以观察每轮参与客户端数量变化对联邦学习性能的影响。 k 值较小时，计算开销较低，通信量较小，训练速度相对更快。但聚合更新方向波动更大，全局模型的准确率提升较慢。 k 较大时，每轮有更多客户端参与训练。服务器每一轮可以接收到更多本地模型更新，聚合结果能更充分反映多个客户端的数据特征。
- 选择 $\text{local_epochs}=1$ 可以避免单个客户端本地训练过多导致模型过度偏向局部数据； $\lambda=0.1$ 用于控制聚合步长，降低参数更新过大带来的不稳定风险。

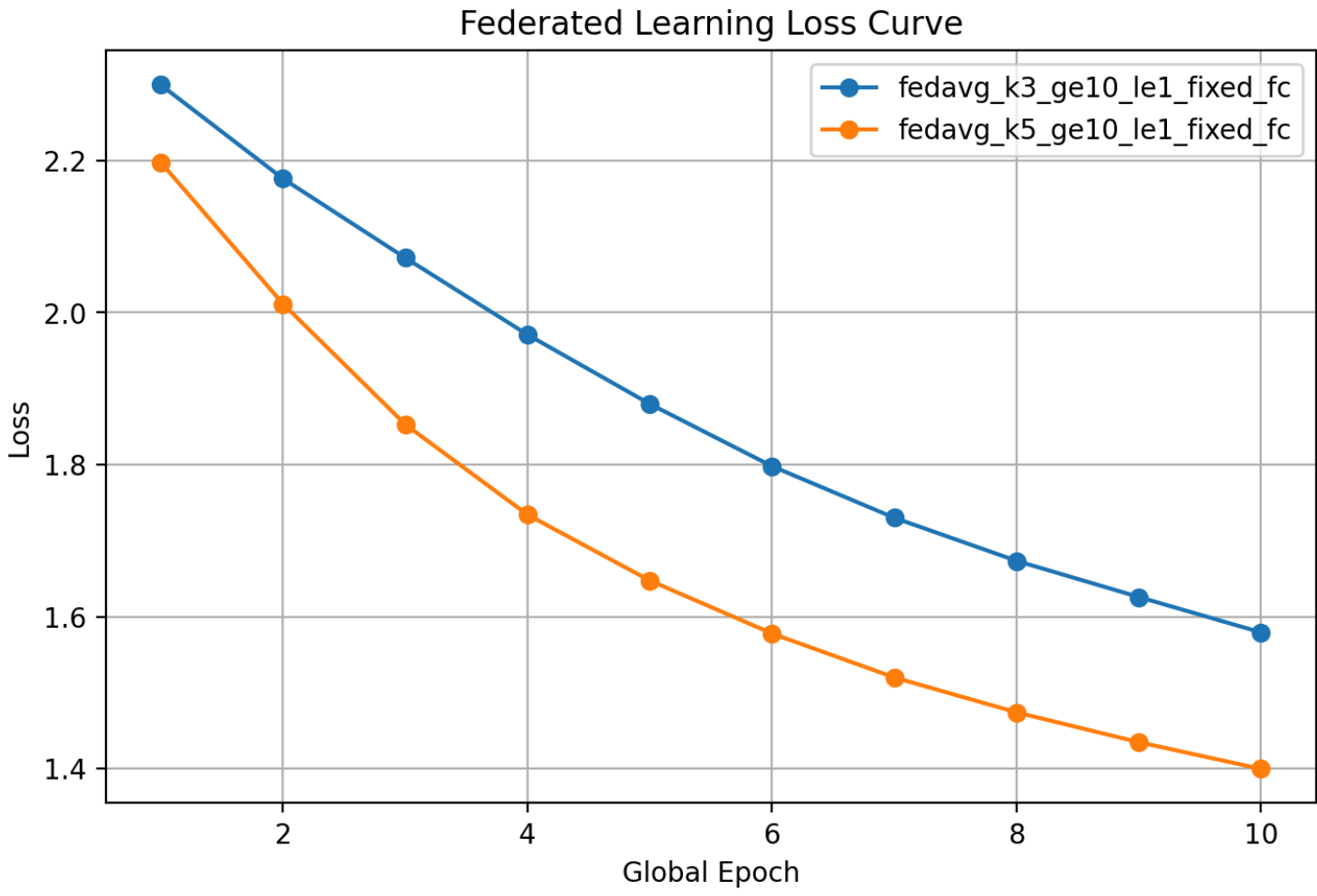
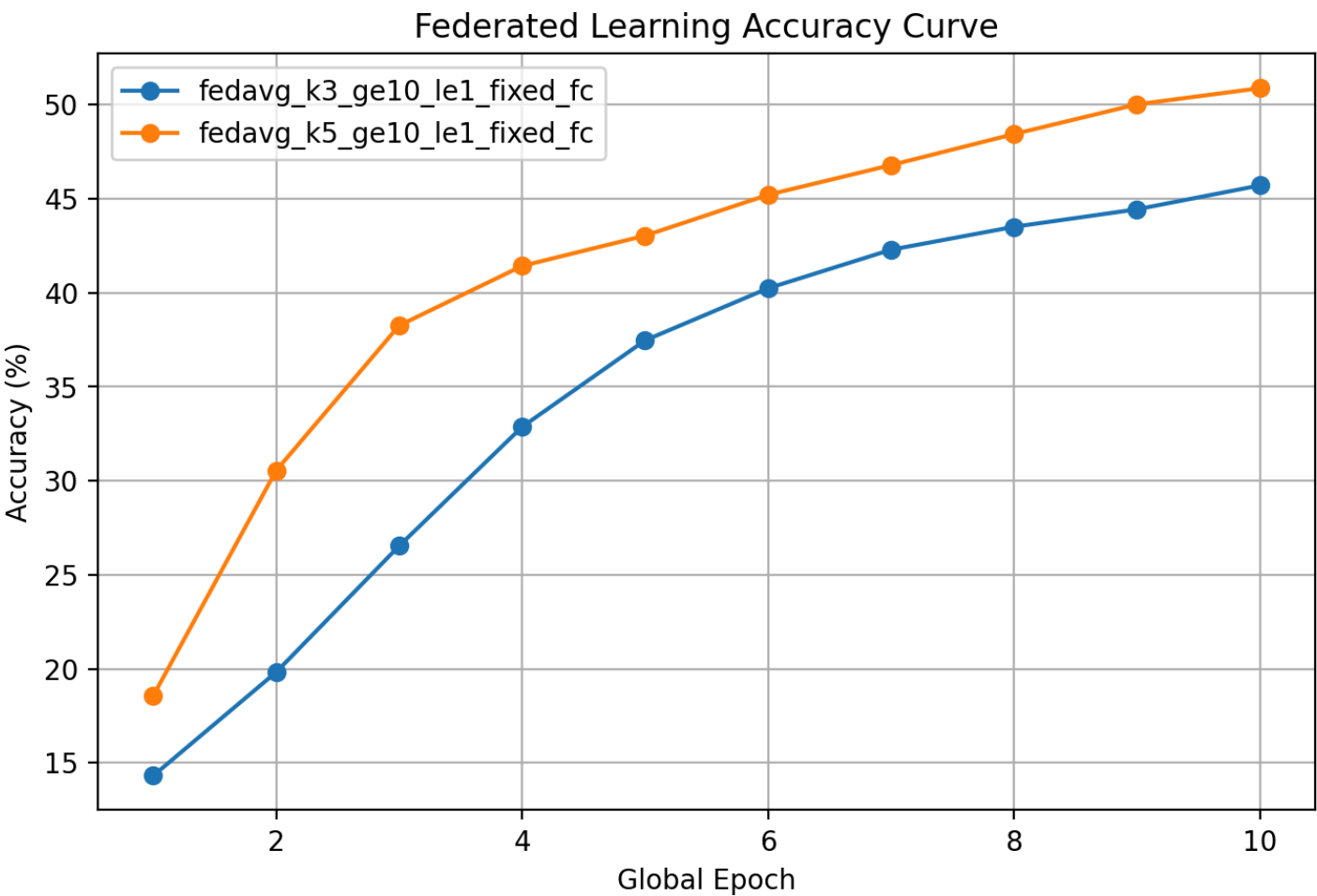
4. 实验结果与分析

4.1 全过程记录

experiment	global_epoch	k	accuracy	loss	selected_clients
k3_ge10	1	3	14.31	2.299608	[1, 0, 4]
k3_ge10	2	3	19.82	2.176221	[3, 9, 2]
k3_ge10	3	3	26.53	2.072054	[1, 8, 9]
k3_ge10	4	3	32.86	1.970995	[9, 6, 0]
k3_ge10	5	3	37.46	1.879843	[0, 1, 3]
k3_ge10	6	3	40.23	1.797929	[3, 8, 0]
k3_ge10	7	3	42.28	1.730059	[8, 3, 6]
k3_ge10	8	3	43.50	1.673433	[3, 7, 4]
k3_ge10	9	3	44.43	1.625865	[0, 2, 6]
k3_ge10	10	3	45.70	1.579267	[5, 4, 2]

experiment	global_epoch	k	accuracy	loss	selected_clients
k5_ge10	1	5	18.56	2.197680	[1, 0, 4, 9, 6]
k5_ge10	2	5	30.54	2.011396	[2, 1, 8, 4, 3]
k5_ge10	3	5	38.25	1.852869	[0, 9, 1, 7, 6]
k5_ge10	4	5	41.42	1.734096	[8, 0, 3, 5, 6]
k5_ge10	5	5	43.03	1.647546	[8, 6, 3, 7, 4]
k5_ge10	6	5	45.20	1.577810	[4, 0, 2, 5, 3]
k5_ge10	7	5	46.78	1.519994	[5, 4, 2, 1, 7]
k5_ge10	8	5	48.43	1.474094	[1, 9, 6, 0, 2]
k5_ge10	9	5	50.01	1.434851	[5, 4, 0, 9, 3]
k5_ge10	10	5	50.87	1.399670	[8, 1, 6, 0, 4]

4.2 训练曲线



从准确率曲线看，k=3 和 k=5 都呈现逐轮上升趋势，说明 FedAvg 聚合确实让全局模型不断吸收客户端本地训练结果。从损失曲线看，两组实验的 loss 都逐轮下降，说明训练过程有效。

4.3 k=3 与 k=5 对比

实验名称	k	最终准确率	最佳准确率	最终损失	最小损失
fedavg_k3_ge10_le1_fixed_fc	3	45.70%	45.70%	1.5793	1.5793
fedavg_k5_ge10_le1_fixed_fc	5	50.87%	50.87%	1.3997	1.3997

k=3 的准确率从第 1 轮的 14.31% 提升到第 10 轮的 45.70%，loss 从 2.2996 降到 1.5793。k=5 的准确率从第 1 轮的 18.56% 提升到第 10 轮的 50.87%，loss 从 2.1977 降到 1.3997。

对比可见，k=5 在最终准确率和最终损失上都优于 k=3。原因是每轮参与聚合的客户端更多，全局更新综合了更多本地数据分布，参数方向更稳定，对全局测试集的泛化更好。但相应地，k=5 每轮需要更多客户端训练和上传参数，通信与计算成本高于 k=3。

4.4 联邦学习与集中式学习的差异

与集中式训练相比，联邦学习每一轮只使用部分客户端上传的参数更新，而不是直接在完整训练集上做统一梯度下降。因此，联邦学习的收敛通常更慢，且更容易受到客户端数据分布差异影响。但是，联邦学习的优势在于原始数据不离开客户端，适用于隐私敏感或数据无法集中存储的场景。

本实验中的准确率没有达到集中式 ResNet18 在 CIFAR-10 上可能达到的较高水平，主要原因包括：全局训练轮数较少、本地训练轮数较少、客户端数据被切分、每轮只选择部分客户端、模型没有进行充分调参。这些现象符合联邦学习中“隐私保护和训练效率之间存在权衡”的特点。

5. 结论与反思

本实验实现了一个基本横向联邦学习流程，包括模型下发、本地训练、参数上传和服务端聚合。实验结果表明，在 CIFAR-10 上，FedAvg 风格聚合能够使全局模型准确率逐步提升，loss 持续下降。对比 k=3 和 k=5 可知，每轮参与客户端数量增加后，模型最终性能更好，但同时会增加通信和计算开销。

实验的局限性主要有三点。第一，训练轮数较少，模型尚未充分收敛。第二，数据划分较简单，未系统比较 IID 和 non-IID 情况。第三，只实现了基础 FedAvg，没有与 FedProx、个性化联邦学习或安全聚合方法比较。

后续可以从以下方向改进：增加全局训练轮数，调整学习率和聚合系数；比较不同 k 值和不同本地训练轮数；构造 non-IID 数据划分，观察客户端数据异质性影响；实现 FedProx，通过近端项限制本地模型偏离全局模型，从而提高 non-IID 场景下的稳定性。

附录：加分项

在基础实验外，我引入了 FedProx 算法，与 FedAvg 进行对比。与 FedAvg 相比，FedProx 在本地 loss 里加入一项，让本地模型不要偏离服务器下发的全局模型太远。

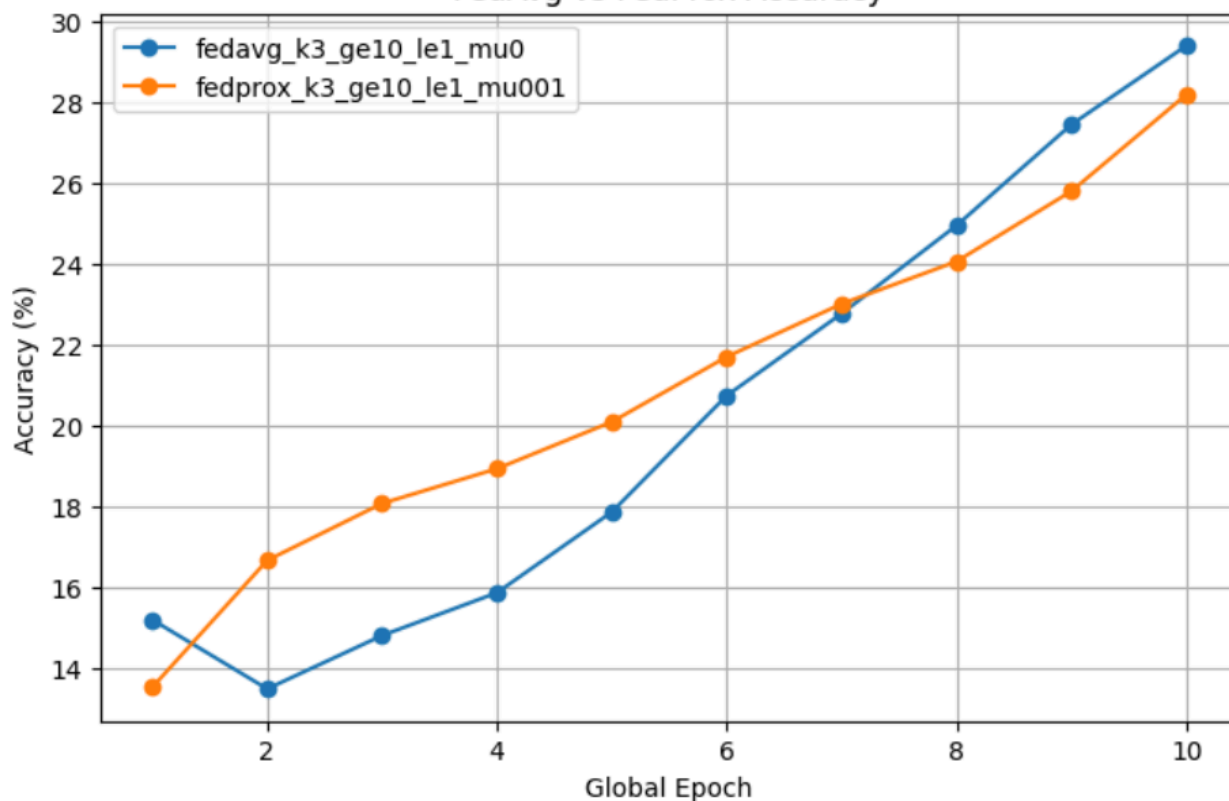
$$\min_w L_i(w) + \frac{\mu}{2} \|w - w_t\|^2$$

其中，参数含义如下：

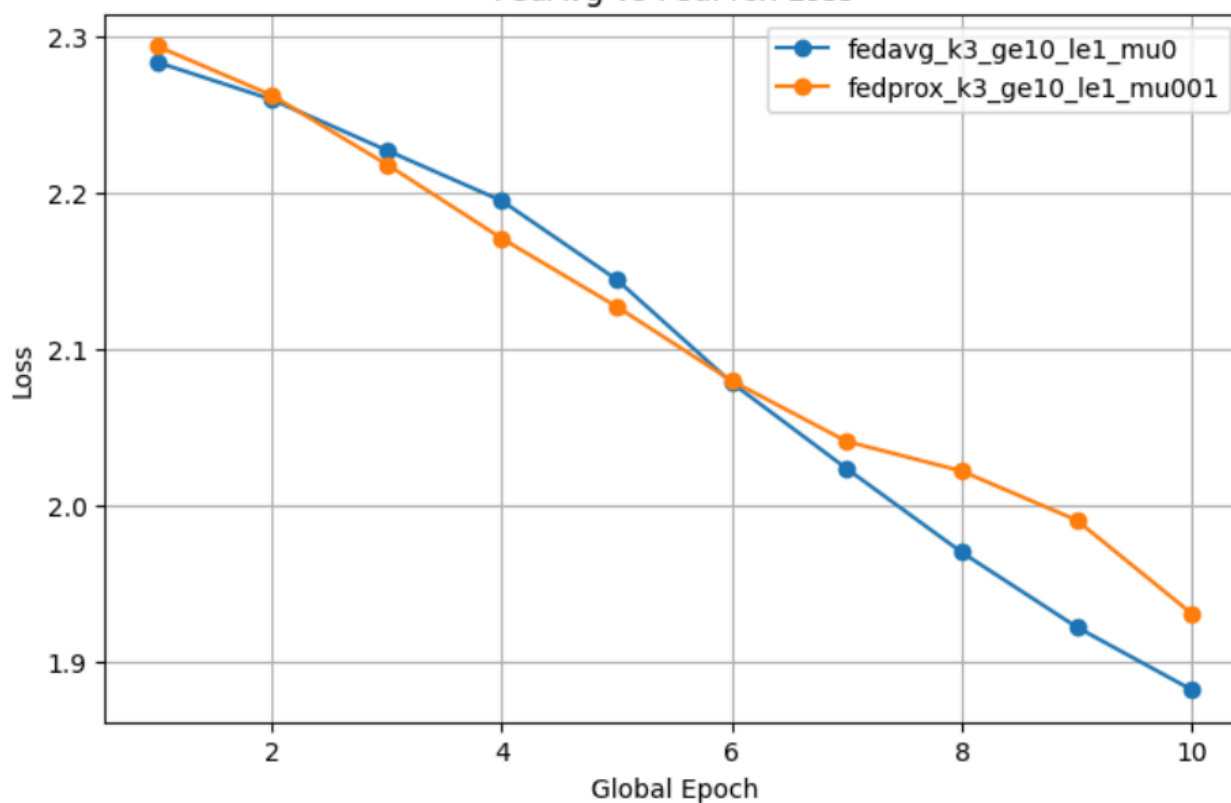
参数	含义
(w)	客户端本地模型参数
(w_t)	当前轮服务器下发的全局模型参数
(\mu)	proximal coefficient，控制约束强度
(\mu=0)	退化为 FedAvg
(\mu>0)	FedProx

运行测试时，使用 k = 3, global_epoch = 10 的配置。对于 FedProx，mu 取 0.01。
在 CIFAR-10 全量数据集下测试结果如下：

FedAvg vs FedProx Accuracy



FedAvg vs FedProx Loss



从 FedAvg 与 FedProx 的准确率曲线可以看出，两种方法的准确率都随着全局训练轮数增加而逐步上升，说明联邦学习训练流程有效。FedProx 在前期训练中表现更平稳，并在第 2 到第 6 轮整体高于 FedAvg，说明 proximal term 对客户端本地更新具有一定约束作用，可以缓解本地模型偏离全局模型的问题。但是在第 7 轮以后，FedAvg 准确率逐渐超过 FedProx，最终 FedAvg 达到约 29.4%，FedProx 达到约 28.2%。这表明在本实验的近似 IID 数据划分、local_epochs=1 和 $\mu=0.01$ 设置下，客户端本地漂移并不严重，因此 FedProx 的优势没有完全体现；相反，proximal term 可能使本地更新更保守，导致最终准确率略低于 FedAvg。